



Lecture 4: Advanced RAG Patterns

Self-RAG, Corrective RAG, Agentic RAG, Graph RAG & More

Part of the RAG Mastery Series



What You Will Learn Today

1. Self-RAG: LLM that decides when to retrieve
 2. Corrective RAG (CRAG): Fixing bad retrieval
 3. Agentic RAG: Multi-step reasoning with retrieval
 4. Graph RAG: Using knowledge graphs for retrieval
 5. Adaptive RAG: Dynamic retrieval strategies
 6. Multimodal RAG: Beyond text
-

1. Self-RAG: The LLM That Thinks Before Retrieving

The Problem with Standard RAG

In standard RAG, you ALWAYS retrieve before generating. But what if:

- The question doesn't need retrieval? ("What is 2+2?")
- The retrieved documents are irrelevant?
- You need to verify the answer after generating?

Self-RAG Solution

Self-RAG trains the LLM to **decide** when and how to retrieve using special tokens.

Standard RAG:

Query → Always retrieve → Always generate with context

Self-RAG:

Query → [Should I retrieve?]

└─ No → Generate directly

└─ Yes → Retrieve → [Is context relevant?]

└─ No → Retrieve again / skip

└─ Yes → Generate → [Is answer supported?]

└─ No → Revise

└─ Yes → Return

answer

Special Tokens in Self-RAG

Token	Meaning
[Retrieve]	Decision: should I retrieve?
[IsRel]	Is the retrieved context relevant?
[IsSup]	Is the answer supported by context?
[IsUse]	Is the answer useful for the question?

Self-RAG Process Example

Query: "What is the capital of France?"

Step 1: [Retrieve] → "No" (LLM knows this from training)

Step 2: Generate → "The capital of France is Paris."

Step 3: [IsSup] → "No external source needed, common knowledge"

Step 4: [IsUse] → "Yes, directly answers the question"

Step 5: Return answer with confidence

Query: "What is our company's 2025 Q3 revenue?"

Step 1: [Retrieve] → "Yes" (needs specific company data)

Step 2: Retrieve relevant documents → Gets financial report

Step 3: [IsRel] → "Yes, document contains Q3 revenue data"

Step 4: Generate → "Q3 2025 revenue was \$45.2M"

Step 5: [IsSup] → "Yes, supported by page 7 of the report"

Step 6: Return answer with citation: "Source: Q3 Report, pg 7"

Why Self-RAG Matters

- **Reduces unnecessary retrievals** (saves cost and time)
- **Improves answer quality** by verifying support
- **Provides citations** (shows where info came from)
- **Handles both parametric and non-parametric knowledge**

2. Corrective RAG (CRAG)

The Problem

Standard RAG blindly uses whatever is retrieved. If retrieval returns bad results, the LLM will give a bad answer.

CRAG Solution

CRAG adds a **correction step** between retrieval and generation.

```
Query → Retrieve → Evaluate Quality →  
  └─ Good: Use as-is  
  └─ Bad: Search the web for better info  
  └─ Ambiguous: Combine retrieval + web search  
→ Generate answer
```

CRAG's Three-State Evaluation

```
def evaluate_retrieval(query, retrieved_docs):  
    scores = []  
    for doc in retrieved_docs:  
        scores.append(relevance_model.score(query, doc))  
  
    avg_score = mean(scores)  
  
    if avg_score > 0.7:  
        state = "SUFFICIENT"          # Retrieved docs are good  
        action = "Use as-is"  
    elif avg_score < 0.3:  
        state = "INSUFFICIENT"        # Retrieved docs are irrelevant  
        action = "Search the web"  
    else:  
        state = "AMBIGUOUS"           # Some docs are good, some aren't  
        action = "Combine retrieval + web search"  
  
    return state, action
```

Web Search as Fallback

When internal documents are insufficient:

```
Query: "Latest Python 3.13 features"  
Internal docs: "Python 3.10 was released in 2021" (outdated!)  
Evaluate: INSUFFICIENT (docs are old)  
Action: Search web → "Python 3.13 released Oct 2024 with..."  
Generate: "Python 3.13, released in October 2024, includes..."
```

Knowledge Refinement in CRAG

CRAG also refines the retrieved knowledge:

1. **Decompose** retrieved documents into fine-grained knowledge strips
2. **Filter** out irrelevant strips
3. **Keep** only relevant strips

Retrieved Document (10 pages):

Page 1: Relevant ✓

Page 2: Irrelevant ✗

Page 3: Partially relevant → Extract relevant sentences ✗✓

Page 4: Irrelevant ✗

Page 5: Relevant ✓

Result: Page 1 + relevant sentences from Page 3 + Page 5
(Clean, focused context instead of all 10 pages)

3. Agentic RAG: Multi-Step Reasoning

The Idea

Instead of one-shot retrieval, use an **LLM agent** that can:

- Plan what to retrieve
- Retrieve, evaluate, and retrieve again if needed
- Use multiple tools (web search, SQL, calculator)
- Synthesize information from multiple sources

Agentic RAG Architecture



Example: Complex Question

User: "Compare the market strategies of Company A and Company B in 2025"

Agent Step 1: Plan

→ "I need info on Company A's strategy and Company B's strategy"

Agent Step 2: Retrieve Company A

→ Search vector DB for "Company A market strategy 2025"

→ Found: "Company A focused on AI products in 2025"

Agent Step 3: Retrieve Company B

→ Search vector DB for "Company B market strategy 2025"

→ Found: "Company B expanded to Asian markets in 2025"

Agent Step 4: Evaluate

→ "I have info on both. But let me check if there's comparison data"

Agent Step 5: Search comparison

→ "Company A vs Company B 2025 comparison"

→ Found analysis report with side-by-side comparison

Agent Step 6: Synthesize

→ "Based on the analysis report..."

→ Generates comprehensive comparison answer

Tools used: Vector DB (2 searches) + Web Search (1 search)

LLM reasoning: 6 steps, 3 retrievals, 1 final answer

LangGraph for Agentic RAG

```
from langgraph.graph import StateGraph

# Define the agent workflow
workflow = StateGraph(AgentState)

workflow.add_node("planner", plan_node)
workflow.add_node("retriever", retrieve_node)
workflow.add_node("evaluator", evaluate_node)
workflow.add_node("generator", generate_node)

workflow.add_edge("planner", "retriever")
workflow.add_edge("retriever", "evaluator")
workflow.add_conditional_edges("evaluator",
    should_continue, # Function: should we retrieve more?
    {"continue": "retriever", "done": "generator"}
)
workflow.add_edge("generator", END)
```

4. Graph RAG: Knowledge Graphs Meet RAG

What is Graph RAG?

Graph RAG uses a **knowledge graph** instead of (or alongside) a vector database.

Knowledge Graph = Entities (nodes) + Relationships (edges)

```
Entities: [Person, Company, Product, Location]
```

```
Relationships:
```

- (Elon Musk) -- [CEO_OF] --> (Tesla)
- (Tesla) -- [PRODUCES] --> (Model 3)
- (Model 3) -- [MADE_IN] --> (Fremont)
- (Fremont) -- [LOCATED_IN] --> (California)

How Graph RAG Works

Step 1: Build Knowledge Graph

Document: "Elon Musk is the CEO of Tesla. Tesla produces the Model 3 in Fremont, California."

LLM extracts:

Entities: Elon Musk (Person), Tesla (Company), Model 3 (Product), Fremont (City), California (State)

Relationships:

- Elon Musk → CEO_OF → Tesla
- Tesla → PRODUCES → Model 3
- Model 3 → MADE_IN → Fremont
- Fremont → LOCATED_IN → California

Step 2: Store in Graph Database

- Neo4j, Amazon Neptune, or any graph DB

Step 3: Query with Graph Traversal

User: "Who is the CEO of the company that makes Model 3?"

Graph Query:

1. Find entity "Model 3"
2. Traverse PRODUCES edge backwards → find "Tesla"
3. Traverse CEO_OF edge from Tesla → find "Elon Musk"

Answer: "Elon Musk is the CEO of Tesla, which produces Model 3."

Graph RAG Advantages

Advantage	Explanation
Multi-hop reasoning	"Who is the CEO of the company that makes Model 3?"
Relationship queries	"What products does Tesla make?"
Structured data	Facts are explicit, not buried in text
Inference	Can discover indirect relationships

Microsoft GraphRAG (Open Source)

Microsoft released GraphRAG in 2024 — it:

- 1. Extracts entities and relationships from documents
- 2. Creates a knowledge graph
- 3. Uses community detection to find clusters
- 4. Generates community summaries
- 5. Uses these summaries for RAG

```
# Microsoft GraphRAG
from graphrag.query import GlobalSearch, LocalSearch

# Local search: specific questions about entities
local_search = LocalSearch(llm, context_builder)
result = local_search.search("What is Elon Musk's role at Tesla?")

# Global search: questions about the whole dataset
global_search = GlobalSearch(llm, context_builder, community_reports)
result = global_search.search("What are the main topics in this dataset?")
```

5. Adaptive RAG

The Idea

Different questions need different retrieval strategies:

Question Type	Best Strategy
Factual (what is X?)	Direct vector search
Multi-hop (why did X cause Y?)	Graph traversal + multi-step retrieval
Current events (latest news?)	Web search
Numerical (what is the revenue?)	SQL query + retrieval

Adaptive RAG Router

```
def route_query(query):  
    classification = llm.classify(query)  
  
    if classification == "factual":  
        return vector_search(query)  
    elif classification == "multi-hop":  
        return graph_traversal(query)  
    elif classification == "current":  
        return web_search(query)  
    elif classification == "numerical":  
        return sql_query(query)  
    elif classification == "summary":  
        return page_level_retrieval(query)  
    else:  
        return hybrid_retrieval(query)
```

6. Multimodal RAG

Beyond Text: Images, Tables, Audio

Multimodal RAG can retrieve and reason over:

- **Images** → Use CLIP embeddings
- **Tables** → Parse and embed table structure
- **Charts/Graphs** → Use vision models to understand
- **Audio** → Transcribe then embed

User: [uploads a chart image] "What trend does this show?"

1. Vision model describes the chart: "Line chart showing revenue growth from 2020-2025, with a sharp increase in 2024"
2. Embed this description + search for related context
3. Retrieve documents about 2024 revenue growth
4. Generate answer: "The chart shows a 40% revenue increase in 2024, primarily driven by the AI product line launch..."

7. Quick Summary

Self-RAG: LLM decides when to retrieve using special tokens

CRAG: Corrects bad retrieval with web search fallback

Agentic RAG: Multi-step reasoning with multiple tools

Graph RAG: Use knowledge graphs for relationship queries

Adaptive RAG: Route queries to the best retrieval strategy

Multimodal RAG: Handle images, tables, audio alongside text

Key Insight: Modern RAG is not one technique – it's a combination of specialized strategies for different question types.



Homework

1. Implement CRAG: retrieve → evaluate → web search if needed → generate
2. Build a simple knowledge graph from 5 documents using spaCy NER
3. Implement a query router that classifies queries and uses different strategies
4. Explore Microsoft GraphRAG on GitHub

Next Lecture: TurboVec Integration — How to use the vector index you discovered in your RAG pipeline